

DA3408 Assignment 2

Pakshal Nagda

September 19, 2026

1 Single Stage vs Multi Stage Docker

1. The naive classifier image size is **2191 MB**.
2. The multi-stage classifier image size is **833 MB**.
This corresponds to a **61.98 %** reduction of disk usage.

```
(venv) pakshal@Laptop:~/lab/assn2$ docker images spam-classifier
```

IMAGE	ID	DISK USAGE	CONTENT SIZE
spam-classifier:multistage	6555a4cbb2ec	833MB	193MB
spam-classifier:naive	e44027cae33d	2.14GB	530MB

3. I trained the Multinomial Naive Bayes model while building the docker image.¹ In case of a single-stage build, this forces me to keep unnecessary files and codes (like data generation and training) in the final image even though their usage is only limited till the training stage. However, in multi-stage build, I use these codes only for training stage and copy only the trained model to the final stage, thus saving space. Further, I can use `python:3.11-slim` for the final stage, leading to further reduction of space.

2 Multi-container Orchestration with Docker Compose

1. `app.py` was modified such that for every text, it checks the Redis cache first and, in case of a miss, predicts using the model and stores the result in the Redis cache with a TTL of **300 seconds**.
2. The required `docker-compose.yml` file was created that maps the predictor app to port 8000.
3. A python script takes user input, sends it to the predictor server and print the output along with the time taken. It can be seen from the results (figure on the right) that while the time taken for new text is > 10 sec, the time taken for repeated text is < 8 sec. Thus the cache does measurably speed up the request on second call.
4. Docker Compose solves the problem of defining and running multiple related and inter-dependent containers together with a simpler way to handle inter-container networking and resource sharing. Docker Compose is intended for coordinating multi-container applications in a single host machine, whereas Kubernetes is a larger orchestration platform designed to manage containerized applications across clusters, including features such as scaling, service discovery, rolling updates, and self-healing.

```
pakshal@Laptop:~/lab/assn2/q2$ python test_perf.py
Spam detector started.
Enter a sentence to classify.
Press Ctrl+C to exit.

Enter text: Play basketball tonight.
Prediction : ham
Time taken : 12.16 ms

Enter text: Free money offer.
Prediction : spam
Time taken : 10.28 ms

Enter text: Free money offer.
Prediction : spam
Time taken : 7.26 ms

Enter text: Play basketball tonight.
Prediction : ham
Time taken : 7.42 ms

Enter text: Play basketball tonight.
Prediction : ham
Time taken : 7.96 ms

Enter text: ^C
Exiting...
```

3 Kubernetes Indexed Job: Parallel Data Validation

1. Since I have 2 nodes with 2 CPUs each, I select **parallelism = $2 \times 2 = 4$** . This is because processing each csv can be done on a single CPU. Since we have 4 CPUs available concurrently, the maximum parallelism is 4 without oversubscribing the cluster.

¹Although in real-world deployments, we do not train models during the build stage, but it was done here to highlight the advantages of multi-stage build. See more details in the README in github.

- It is visible from the output of the `get pods` command that the intended parallelism was achieved. Two pods on each node started together 5s ago and 1s ago. Thus, parallelism of 4 was followed.

```
pakshal@Laptop:~/lab/assn2/q3$ kubectl get pods -l app=signup-validator -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
signup-validator-0-tvqjd	0/1	Completed	0	5s	10.244.1.11	minikube-m02
signup-validator-1-spmwx	0/1	Completed	0	5s	10.244.0.19	minikube
signup-validator-2-x289w	0/1	Completed	0	5s	10.244.1.10	minikube-m02
signup-validator-3-v8w2h	0/1	Completed	0	5s	10.244.0.20	minikube
signup-validator-4-zqn6s	0/1	Completed	0	1s	10.244.1.13	minikube-m02
signup-validator-5-2bk2v	0/1	Completed	0	1s	10.244.0.21	minikube
signup-validator-6-f5z95	0/1	Completed	0	1s	10.244.1.12	minikube-m02
signup-validator-7-ndl28	0/1	Completed	0	1s	10.244.0.22	minikube

- The pod logs below show the logs (print statements) from each pod. This shows that the downward API was correctly configured, as the node names and pod names are correct. It also shows the invalid rows found in each of the shards.

```
pakshal@Laptop:~/lab/assn2/q3$ for pod in $(kubectl get pods -l app=signup-validator -o jsonpath='{.items[*].metadata.name}'); do kubectl logs $pod; done
node=minikube-m02 | pod=signup-validator-0-tvqjd | shard=0 | total_rows=100 | invalid_rows=19
node=minikube | pod=signup-validator-1-spmwx | shard=1 | total_rows=100 | invalid_rows=26
node=minikube-m02 | pod=signup-validator-2-x289w | shard=2 | total_rows=100 | invalid_rows=17
node=minikube | pod=signup-validator-3-v8w2h | shard=3 | total_rows=100 | invalid_rows=15
node=minikube-m02 | pod=signup-validator-4-zqn6s | shard=4 | total_rows=100 | invalid_rows=22
node=minikube | pod=signup-validator-5-2bk2v | shard=5 | total_rows=100 | invalid_rows=18
node=minikube-m02 | pod=signup-validator-6-f5z95 | shard=6 | total_rows=100 | invalid_rows=18
node=minikube | pod=signup-validator-7-ndl28 | shard=7 | total_rows=100 | invalid_rows=19
```

Using a shared volume to log pods is not preferred here since by default a shared storage will be in ReadWriteOnce (RWO) access mode, so multiple concurrent pods will not be able to write their outputs simultaneously in the shared volume.² The kubectl API allows us to inspect logs of each pod individually, which is desirable.

- Yes, if we have a total of 6 CPUs available, I will **change parallelism to 6**. Justification is just that it is best to utilize all the resources we have because the task is embarrassingly parallelizable. Although this will also require two waves (6 + 2) to complete all tasks like earlier (4 + 4), it is more robust to scaling of the number of shards.

4 Kubernetes Deployments: Self-Healing & Rolling Updates

```
pakshal@Laptop:~/lab/assn2/q4$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
spam-classifier-deployment-b7bfd485f-gfr2m	1/1	Running	0	20s
spam-classifier-deployment-b7bfd485f-h7hbf	1/1	Running	0	20s

- The **ReplicaSet controller** (managed by the Deployment controller) is responsible for self-healing. It decides a replacement is needed by **continuously running a control loop that compares the spec-defined state (replicas: 2) against the actual cluster state**. When a pod is deleted, the count drops to 1, causing ReplicaSet to spawn a new pod to restore the count to 2.

```
pakshal@Laptop:~/lab/assn2/q4$ kubectl delete pod spam-classifier-deployment-b7bfd485f-gfr2m
pod "spam-classifier-deployment-b7bfd485f-gfr2m" deleted from default namespace
pakshal@Laptop:~/lab/assn2/q4$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
spam-classifier-deployment-b7bfd485f-6kx26	0/1	Running	0	5s
spam-classifier-deployment-b7bfd485f-h7hbf	1/1	Running	0	3m8s

- If deployed as a Kubernetes Job, updating the image spec would **not trigger a zero-downtime rolling update**. Jobs execute pods to completion and terminate. Updating a Job manifest directly is disallowed by Kubernetes without deleting and recreating the Job resource entirely, which would result in complete service downtime during the transition.

```
pakshal@Laptop:~/lab/assn2/q4$ kubectl rollout status deployment/spam-classifier-deployment
Waiting for deployment "spam-classifier-deployment" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "spam-classifier-deployment" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "spam-classifier-deployment" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "spam-classifier-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "spam-classifier-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "spam-classifier-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "spam-classifier-deployment" successfully rolled out
pakshal@Laptop:~/lab/assn2/q4$ kubectl rollout history deployment/spam-classifier-deployment
deployment.apps/spam-classifier-deployment
REVISION  CHANGE-CAUSE
1          <none>
2          <none>
```

- Deployments** are designed for long-running, stateless services (like model serving and web APIs). They maintain a target replica count via self-healing during pod failures and support zero-downtime rolling updates via readiness probes to keep services continuously available to users.
 - Jobs** are designed for finite, run-to-completion tasks (like data validation or model training). They run pods until a successful exit code is returned and track completion metrics across shards, making them unsuitable for continuous traffic serving.

²There is a way to change the access mode to ReadWriteMany (RWX), but that is unnecessary for this assignment.